

On the Lavellian Tracker Derivative

Julian Lavelle

September 6, 2026

Abstract

In cryptography, side-channels are a category of security vulnerabilities that are based around information given to an attacker by the implementation of an algorithm. Through side channel attacks, secrets can be recovered by simply timing the operations or listening to the computer's fan.

In this paper, we'll be discussing side-channel attacks and using the Lavellian Tracker Derivative (LTD) as a defense against them. In using the LTD, we'll be able to extract secret information from algorithms, block side-channel attacks, and overall assess the security of cryptographic algorithms. We'll also introduce a set of operators called the Lavellian Algorithmic operators, which allow us to mathematically write cryptographic algorithms with ease.

Section 1: The LTD

1.1 The Tracking Operator

To begin, we define the tracking operator $\hat{T}[f]$, which tracks the cost an operation f took to complete. This cost can be time, power, audio, or any other observable metric on f . Note that this includes all costs; for example, CPU noise is included within $\hat{T}$. It is the literal cost of that operation, not the abstract one.

$\hat{T}$ has one rule; the cost taken by $f(g(x))$ is the cost taken by f to compute on g and g to compute on x :

$$\hat{T}[f(g(x))] = \hat{T}[f](g(x)) + \hat{T}[g](x)$$

When it's not applicable to write a function in named notation, we use $T[\text{operator}]$, like in the example below:

$$\hat{T}[ax + b] = \hat{T}[+](ax, b) + \hat{T}[*](a, x)$$

where the asterisk denotes multiplication. This result represents the total cost of completing the operation $ax + b$, which is the sum of costs for addition and multiplication.

Note that in some cases, the parameters themselves need to be expanded as well; in that case, our above composition rule reads:

$$\hat{T}[f(g(h))] = \hat{T}[f](g(h)) + \hat{T}[g](h) + \hat{T}[h]$$

1.2 Lavellian Tracker Derivatives

To check the variation in $\hat{T}$ upon changing variable x , we'll take the Lavellian Tracker Derivative, denoted as:

$$\frac{\partial \hat{T}}{\partial x} = \hat{T}'_x$$

When cost varies with a parameter, $\hat{T}'_x$ will be nonzero; otherwise, $\hat{T}'_x$ will be zero. Before using the LTD, it's important to set up the environment. We'll define the LTD of each atomic operation we use, either definitely (with a value) or indefinitely (without a value), like below:

$$\hat{T}'_a[+](a, b) = \hat{T}'_b[+](a, b) = 1, \quad \hat{T}'_x[\sin](x) \neq 0, \quad \hat{T}'_x[\ln](x) = 0$$

where the first LTD is definite (variation of 1 per unit change in a or b), the second is indefinite (unset variation per unit change in x), and the third is zero.

To execute a side-channel attack, the attacker must infer a secret from public observables. To do so, the observable must vary with the secret; otherwise, the data recovered is useless. The LTD tells you exactly whether the observable cost varies, and by how much. The tracker itself can be used to glean the secret, while the LTD can be used to glean information assisting your search, such as search resolution (high LTD means less tests are needed to glean the secret).

The LTD then directly correlates to the difficulty of an attack; if the LTD is zero, there is no side channel attack possible. If it's small, many measurements are needed to distinguish secrets during the search. If it's large, less measurements are needed and the search can be done efficiently.

Section 2: Using the LTD

2.1 Attacking

The LTD directly allows us to analyze potential side channel attacks in an algorithm. Take the following function as an example:

$$f(x, y, s) = (x \gg y)^s$$

where $\gg$ is a right bit shift. Suppose our environment is given by:

$$\hat{T}'_a[\gg](a, b) = 0, \quad \hat{T}'_b[\gg](a, b) \neq 0, \quad \hat{T}'_b[\text{pow}](b, e) = 0 \quad \hat{T}'_e[\text{pow}](b, e) \neq 0$$

where pow refers to the exponentiation operation. This says that in the right shift operation, the number of bits shifted affects the cost; in the exponentiation operation, the exponent affects the cost. This could also be written as:

$$\hat{T}[>>](a, b) = t_1(b) \quad \hat{T}[\text{pow}](b, e) = t_2(e)$$

Then:

$$\hat{T}'_b[>>](a, b) = t'_1(b) \quad \hat{T}'_e[\text{pow}](b, e) = t'_2(e)$$

Now we can take the tracker of f :

$$\hat{T}[f] = \hat{T}[(x >> y)^s] = \hat{T}[\text{pow}](x >> y, s) + \hat{T}[>>](x, y)$$

Then the LTD, with respect to each variable:

$$\hat{T}'_x[f] = \hat{T}'_x[\text{pow}](x >> y, s) + \hat{T}'_x[>>](x, y)$$

The first derivative is zero, as the shift of pow of values $x >> y$ and s doesn't have cost variation with x , only possibly $x >> y$ and s ; the second is zero as well, because $\hat{T}'_a[>>](a, b) = 0$. so:

$$\hat{T}'_x[f] = 0$$

Next, we'll take the LTD w.r.t y :

$$\hat{T}'_y[f] = \hat{T}'_y[\text{pow}](x >> y, s) + \hat{T}'_y[>>](x, y)$$

Like before, the first derivative is zero, but this time $\hat{T}'_b[>>](a, b) = t'_1(b)$, so:

$$\hat{T}'_y[f] = t'_1(y)$$

Finally, taking the LTD w.r.t s :

$$\hat{T}'_s[f] = \hat{T}'_s[\text{pow}](x >> y, s) + \hat{T}'_s[>>](x, y)$$

The first derivative is nonzero, as $\hat{T}'_e[\text{pow}](b, e) = t'_2(e)$; the second is zero as $x >> y$ contains no s , so:

$$\hat{T}'_s[f] = t'_2(s)$$

Assembling our results:

$$\hat{T}'_x[f] = 0 \quad \hat{T}'_y[f] = t'_1(y) \quad \hat{T}'_s[f] = t'_2(s)$$

This means that while x is non-recoverable, y and s can be extracted from f . More specifically, this result tells us that for a unit change in y at y , we get a cost change of $t'_1(y)$; per unit change in s at s , we get a cost change of $t'_2(s)$. This implies that a side-channel attack is possible.

2.2 Lavellian Algorithmic Operators

Before we begin our next topic, it is imperative for cleanliness that we find a way to represent cryptographic algorithms mathematically. As it stands, we are currently missing two key programming operations: if statements and loops.

The if statement will be handled by the algorithmic if operator Ξ :

$$a \underset{\text{condition}}{\Xi} b$$

The algorithmic if operator returns it's left-hand value if the condition underneath is true, and the right hand value if the condition is false. In code, the algorithmic if is equivalent to, in psuedocode:

```
if (condition) return a
if (!condition) return b
```

Algorithmic ifs make formerly awkward expressions easy to create:

$$\frac{1}{b} \approx \frac{1}{m} \cdot \sum_i h^i \cdot \left(1 \underset{i \bmod 2=0}{\Xi} -1 \right)$$

The derivative of an algorithmic if w.r.t a variable not in the condition is:

$$\frac{\partial}{\partial x} \left(a \underset{\text{condition}}{\Xi} b \right) = \frac{\partial a}{\partial x} \underset{\text{condition}}{\Xi} \frac{\partial b}{\partial x}$$

Our other operator is the algorithmic aggregate Λ :

$$\underset{\lambda_{-1}=\text{initial}}{\overset{i \in \text{range}}{\Lambda}} f(i, \lambda)$$

Algorithmic aggregates iterate over i and continuously update λ using a body function f . In code:

```
lambda = initial
for i in range: lambda = f(i, lambda)
return lambda
```

The algorithmic aggregate can be switched into a compound form as well; the cryptographic `modpow` function is, for example:

$$\text{modpow}(a, e, n) = \underset{\substack{\lambda_{-1}=1 \\ b_{-1}=a \bmod n}}{\overset{i \in [0, \text{bitlen}(e)]}{\Lambda}} \begin{cases} \lambda = \lambda \cdot b \bmod n & \underset{e_i=1}{\Xi} \lambda \\ b = b^2 \bmod n \end{cases}$$

Which is equivalent to:

```
lambda = 1
b = a % n
for i from 0 to bitlen(e):
    if (e[i] == 1) lambda = lambda * b % n
    b = b^2 % n
return lambda
```

Note that only λ , the first variable, is returned as a result. Derivatives through a algorithmic aggregate are handled via the chain rule:

$$\frac{\partial}{\partial x} \left(\bigwedge_{\lambda_{-1}=\text{initial}}^{i \in \text{range}} f(i, \lambda, x) \right) = \bigwedge_{\substack{i \in \text{range} \\ \lambda'_{-1}=0 \\ \lambda_{-1}=\text{initial}}} \left\{ \begin{array}{l} \lambda' = \frac{\partial f}{\partial \lambda} \cdot \lambda' + \frac{\partial f}{\partial x} \\ \lambda = f(i, \lambda, x) \end{array} \right.$$

where λ' (the first variable) is returned.

2.3 The Kocher Attack

The Rivest-Shamir-Adleman (RSA) encryption algorithm was famously attacked by Paul Kocher in 1996 using a side-channel attack that allowed him to reconstruct secret keys based on the timing of the `modpow` operation. An old RSA implementation would look like this:

```
decrypt(c, d, n):
    result = 1
    base = c % n

    for i from 0 to bitlen(d):
        if (d[i] == 1) result = (result * base) % n
        base = base^2 % n

    return result
```

where c is the ciphertext, d is the secret, and n is a public parameter. In algorithmic form:

$$\text{decrypt}(c, d, n) = \bigwedge_{\substack{i \in [0, \text{bitlen } d] \\ \lambda_{-1}=1 \\ b_{-1}=c \bmod n}} \left\{ \begin{array}{l} \lambda = \lambda \cdot b \bmod n \quad \Xi \quad \lambda \\ b = b^2 \bmod n \end{array} \right.$$

Setting up the environment:

$$\begin{aligned} \hat{T}'_a[\text{mod}](a, b) &= \hat{T}'_b[\text{mod}](a, b) = 0 & \hat{T}'_x[\text{sq}](x) &= 0 \\ \hat{T}'_c \left[\frac{\Xi}{c} \right] (a, b) &\neq 0 & \hat{T}'_a \left[\frac{\Xi}{c} \right] (a, b) &= \hat{T}'_b \left[\frac{\Xi}{c} \right] (a, b) = 0 \end{aligned}$$

We can take the tracker of `decrypt` by simply adding a T variable to our compound algorithmic aggregate:

$$\hat{T}[\text{decrypt}](c, d, n) = \bigwedge_{\substack{i \in [0, \text{bitlen } d] \\ T=0 \\ \lambda_{-1}=1 \\ b_{-1}=c \bmod n}} \left\{ \begin{array}{l} T = T + \hat{T}[\lambda] + \hat{T}[b] \\ \lambda = \lambda \cdot b \bmod n \quad \Xi \quad \lambda \\ b = b^2 \bmod n \end{array} \right.$$

Now, naively this looks like it shouldn't have any side-channel vulnerabilities; the environment seems mostly safe. However, when we take $\hat{T}[\lambda]$:

$$\hat{T}[\lambda] = \hat{T} \left[\lambda \cdot b \bmod n \quad \Xi \quad \lambda \right] = \hat{T}[\lambda \cdot b \bmod n] \quad \Xi \quad 0$$

with that last step coming from the fact that the cost of an algorithmic if is the cost of the branch that fired. Notice that this result means that when we take $\hat{T}'_d$:

$$\hat{T}'_e[\lambda] = \hat{T}'_d \left[\begin{array}{c} \Xi \\ d_i=1 \end{array} \right] (\lambda \cdot b \bmod n, \lambda)$$

This is nonzero, as changing d changes d_i , changing the algorithmic if's outcome. This implementation is vulnerable to a side-channel attack.

The fix, called a **select** function, computes $\lambda \cdot b \bmod n$ anyway, whether or not $d_i = 1$. This changes $\hat{T}'_c \left[\begin{array}{c} \Xi \\ c \end{array} \right] (a, b) \neq 0$ to $\hat{T}'_c \left[\begin{array}{c} \Xi \\ c \end{array} \right] (a, b) = 0$ and kills the vulnerability.

Conclusions

In this paper, we've discussed the utility of the Lavellian Tracker Derivative and used it in sample contexts to identify vulnerabilities in cryptographic algorithms. Using the LTD and the Lavellian Algorithmic Operators, we constructed an RSA implementation and assessed it to reproduce the famous Kocher attack and identified the fix based on our LTD environment. Overall, the LTD proves a valuable tool in the belt of any security analyst working with novel cryptographic algorithms.